

Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский
Университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина «Основы программной инженерии»

Лабораторная работа №3

Вариант №8673 + gradle

Выполнили: Гладышев Иван, Хлопцов Сергей

Группа: P3213

Принимающий работу: Воронина Дарья Сергеевна

Санкт-Петербург 2026

Содержание

Задание.....	3
Выполнение работы	3
Вывод	10

Задание

Написать сценарий для утилиты [Apache Ant \(+ gradle\)](#), реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из [лабораторной работы №3](#) по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запускаемом классе.

Сценарий должен реализовывать следующие цели (targets):

1. **compile** -- компиляция исходных кодов проекта.
2. **build** -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели **compile**.
3. **clean** -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. **test** -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель **build**).
5. **music** - воспроизведение музыки по завершению сборки (цель **build**).
6. **history** - если проект не удаётся скомпилировать (цель **compile**), загружается предыдущая версия из репозитория git. Операция повторяется до тех пор, пока проект не удастся собрать, либо не будет получена самая первая ревизия из репозитория. Если такая ревизия найдена, то формируется файл, содержащий результат операции diff для всех файлов, измененных в ревизии, следующей непосредственно за последней работающей.

Выполнение работы

Созданные команды для Apache Ant:

```
ant compile
ant build
ant test
ant clean
ant music
ant history
```

Главный конфигурационный файл для Apache Ant build.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="PrepareWeb3Ant" default="build" basedir="."
xmlns:ivy="antlib:org.apache.ivy.ant">
    <property file="build.properties"/>
    <property environment="env"/>

    <available file="${music.file}" property="music.file.exists"/>

    <path id="main.classpath"/>
    <path id="test.classpath"/>

    <target name="init">
        <mkdir dir="${build.dir}"/>
        <mkdir dir="${classes.dir}"/>
        <mkdir dir="${test.classes.dir}"/>
        <mkdir dir="${test.reports.dir}"/>
        <mkdir dir="${dist.dir}"/>
        <mkdir dir="${history.output.dir}"/>
        <mkdir dir="${lib.dir}"/>
    </target>

    <target name="download-ivy" depends="init"
unless="ivy.downloaded">
        <available file="${ivy.jar}" property="ivy.downloaded"/>
        <get src="${ivy.url}" dest="${ivy.jar}"
skipexisting="true"/>
    </target>

    <target name="resolve" depends="download-ivy">
        <taskdef resource="org/apache/ivy/ant/antlib.xml"
uri="antlib:org.apache.ivy.ant">
            <classpath>
                <pathelement location="${ivy.jar}"/>
            </classpath>
        </taskdef>

        <ivy:retrieve conf="main"
pattern="${lib.dir}/main/[artifact]-[revision](-
[classifier]).[ext]"/>
        <ivy:retrieve conf="test"
pattern="${lib.dir}/test/[artifact]-[revision](-
[classifier]).[ext]"/>

        <path id="resolved.main.classpath">
            <fileset dir="${lib.dir}/main" includes="*.jar"/>
        </path>
        <path id="resolved.test.classpath">
            <path refid="resolved.main.classpath"/>
            <fileset dir="${lib.dir}/test" includes="*.jar"/>
        </path>
    </target>
</project>
```

```

        </path>
    </target>

    <target name="copy-resources" depends="init">
        <copy todir="${classes.dir}" failonerror="false">
            <fileset dir="${resources.dir}"/>
        </copy>
    </target>

    <target name="prepare-manifest" depends="init">
        <manifest file="${manifest.file}">
            <attribute name="Manifest-Version" value="1.0"/>
            <attribute name="Main-Class" value="${main.class}"/>
            <attribute name="Implementation-Version"
value="${project.version}"/>
        </manifest>
    </target>

    <target name="compile" depends="init,resolve,copy-resources">
        <javac srcdir="${src.dir}"
            destdir="${classes.dir}"
            includeantruntime="false"
            encoding="UTF-8"
            debug="true"
            source="17"
            target="17">
            <classpath refid="resolved.main.classpath"/>
        </javac>
    </target>

    <target name="build" depends="compile,prepare-manifest">
        <jar destfile="${dist.jar}" manifest="${manifest.file}">
            <fileset dir="${classes.dir}"/>
        </jar>
    </target>

    <target name="clean">
        <delete dir="${build.dir}"/>
        <delete dir="${dist.dir}"/>
        <delete dir="${history.output.dir}"/>
    </target>

    <target name="compile-tests" depends="build">
        <javac srcdir="${test.src.dir}"
            destdir="${test.classes.dir}"
            includeantruntime="false"
            encoding="UTF-8"
            debug="true"
            source="17"
            target="17">
            <classpath>
                <pathelement location="${classes.dir}"/>
            </classpath>
        </javac>
    </target>

```

```

        <path refid="resolved.test.classpath"/>
    </classpath>
</javac>
</target>

<target name="test" depends="compile-tests">
    <java
classname="org.junit.platform.console.ConsoleLauncher" fork="true"
failonerror="true">
        <classpath>
            <pathelement location="${classes.dir}"/>
            <pathelement location="${test.classes.dir}"/>
            <path refid="resolved.test.classpath"/>
        </classpath>
        <arg value="--scan-class-path"/>
        <arg
value="${classes.dir}${path.separator}${test.classes.dir}"/>
        <arg value="--reports-dir"/>
        <arg value="${test.reports.dir}"/>
    </java>
</target>

<target name="music" depends="build,music-play,music-skip"/>

<target name="music-play" if="music.file.exists">
    <exec executable="${music.command}" failonerror="true">
        <arg value="${music.file}"/>
    </exec>
</target>

<target name="music-skip" unless="music.file.exists">
    <echo message="Music file ${music.file} not found; skipping
playback."/>
</target>

<target name="history" depends="init">
    <exec executable="/bin/sh" failonerror="true">
        <arg value="scripts/history-check.sh"/>
        <arg value="${basedir}"/>
        <arg value="${git.command}"/>
        <arg value="${ant.command}"/>
        <arg value="${history.worktree.dir}"/>
        <arg value="${history.diff.file}"/>
    </exec>
</target>
</project>

```

Созданные команды для Gradle:

```
gradle compile
gradle build
gradle jar
gradle test
gradle clean
gradle music
gradle history
gradle dance
```

Главный конфигурационный файл для Gradle build.gradle:

```
plugins {
    id 'java'
    id 'war'
}

// грузим из build.properties
def buildProperties = new Properties()
file('build.properties').withInputStream { buildProperties.load(it)
}

group = 'org.example'
version = buildProperties.getProperty('project.version', '1.0-SNAPSHOT')

buildDir = 'target'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}

repositories {
    mavenCentral()
}

sourceSets {
    main {
        java {
            srcDirs = ['java']
        }
        resources {
            srcDirs = ['resources']
        }
    }
    test {
        java {
```

```

        srcDirs = ['test']
    }
}

dependencies {
    implementation 'org.glassfish:jakarta.faces:2.3.9'
    compileOnly 'javax.servlet:javax.servlet-api:4.0.1'
    implementation 'org.primefaces:primefaces:14.0.0'
    implementation 'org.hibernate:hibernate-core:5.6.15.Final'
    implementation 'org.apache.derby:derbyclient:10.16.1.1'

    testImplementation platform('org.junit:junit-bom:5.10.2')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}

tasks.register('dance') {
    dependsOn tasks.named('build')
    group = 'lab3'
    description = 'Танцующий человечек'

    doLast {
        for (int i = 0; i < 12; i++) {
            print "\033[H\033[2J"
            println ""

            (•_•)
            <) )J BUILD
            / \
            ""

                sleep(300)

            print "\033[H\033[2J"
            println ""

            (•_•)
            ʌ( (>
            / \
            ""

                sleep(300)

            }

        }
    }

tasks.named('jar') {
    archiveFileName =
"$${buildProperties.getProperty('project.name', 'PrepareWeb3')}.jar"

```



```

        manifest.from(buildProperties.getProperty('manifest.file',
'MANIFEST.MF'))
    }

tasks.named('war') {
    archiveFileName = 'ROOT.war'
    webAppDirectory = file('webapp')
}

tasks.register('compile') {
    group = 'lab3'
    description = 'Компиляция'
    dependsOn tasks.named('classes')
}

tasks.named('build') {
    dependsOn tasks.named('compile')
    //dependsOn tasks.named('jar')
}

tasks.named('clean') {
    doFirst {
        delete file(buildProperties.getProperty('dist.dir',
'dist'))
        delete
file(buildProperties.getProperty('history.output.dir', 'history-
output'))
    }
}

tasks.register('music', Exec) {
    group = 'lab3'
    description = 'Воспроизведение музыки по завершению сборки'
    dependsOn tasks.named('build')

    println '\nДЗЫНЬ'

    commandLine buildProperties.getProperty('music.command',
'afplay'),
        buildProperties.getProperty('music.file',
'/System/Library/Sounds/Glass.aiff')
}

tasks.register('history', Exec) {
    group = 'lab3'
    description = 'Поиск последней компилируемой ревизии Git и
формирование diff'
    dependsOn tasks.named('clean')
}

```

```
        commandLine 'bash', 'scripts/history-check-gradle.sh'

        environment 'GRADLE_COMMAND',
buildProperties.getProperty('gradle.command', 'gradle')
        environment 'GIT_COMMAND',
buildProperties.getProperty('git.command', 'git')
        environment 'HISTORY_OUTPUT_DIR',
buildProperties.getProperty('history.output.dir', 'history-output')
        environment 'HISTORY_DIFF_FILE',
buildProperties.getProperty('history.diff.file', 'history-
output/last-working-revision.diff')
        environment 'HISTORY_WORKTREE_DIR',
buildProperties.getProperty('history.worktree.dir', 'history-
output/worktree')
    }
```

Остальные файлы / проект целиком можно найти по [ссылке](#)

Вывод

В ходе выполнения лабораторной работы были изучены и реализованы системы автоматической сборки проектов на базе утилит Apache Ant и Gradle.

С использованием Apache Ant был разработан сценарий сборки на языке XML, в котором явно описаны все этапы процесса: компиляция исходного кода, запуск тестов, упаковка в jar-архив, очистка проекта, а также дополнительные цели (воспроизведение музыки и анализ истории изменений). Ant представляет собой императивную систему сборки, в которой разработчик вручную задаёт последовательность действий и зависимости между ними.

С использованием Gradle была реализована аналогичная функциональность, но с применением декларативного подхода. Благодаря плагину java Gradle предоставляет встроенный жизненный цикл сборки (compile, test, build), что существенно упрощает конфигурацию проекта. Дополнительно были настроены пользовательские задачи, включая music и history. Зависимости проекта автоматически загружаются из репозитория Maven Central.

В ходе работы также были изучены принципы модульного тестирования с использованием фреймворка JUnit 5 и его интеграция в системы сборки.